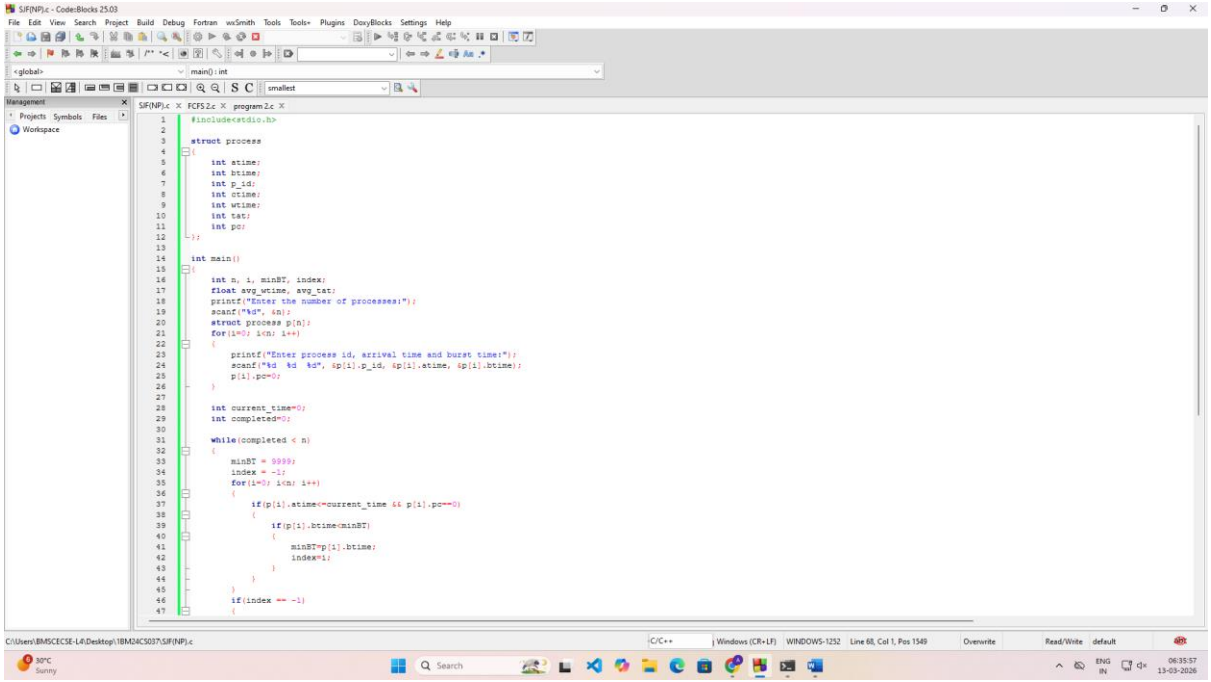
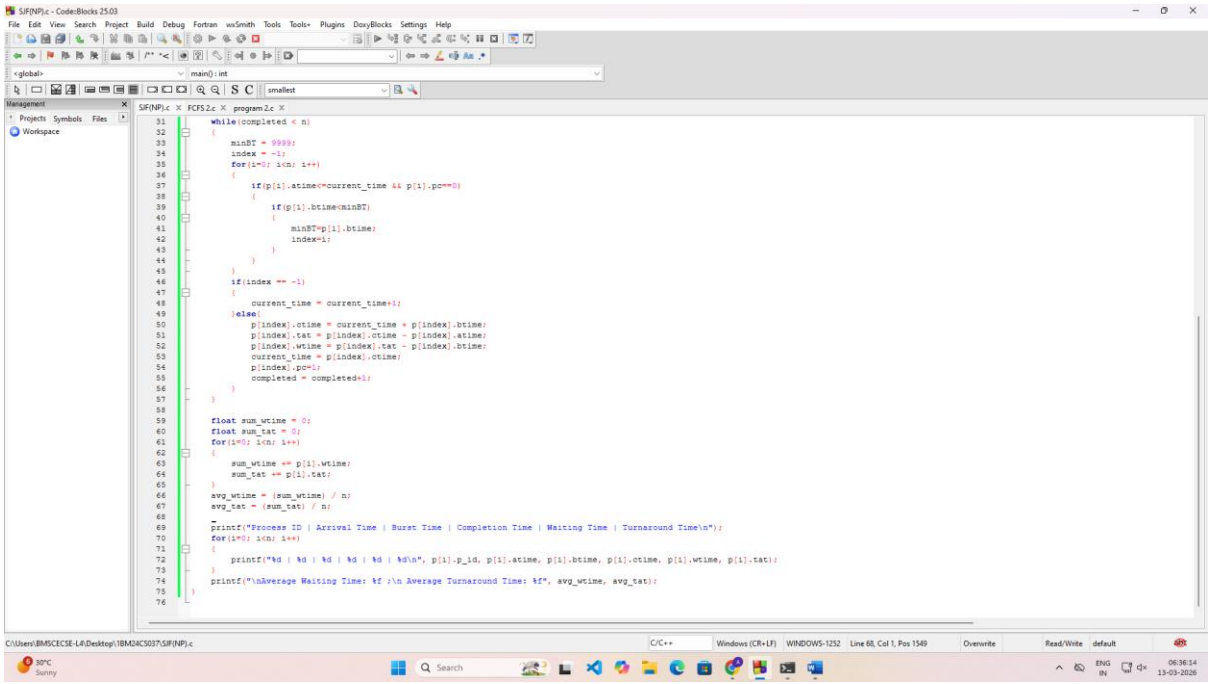


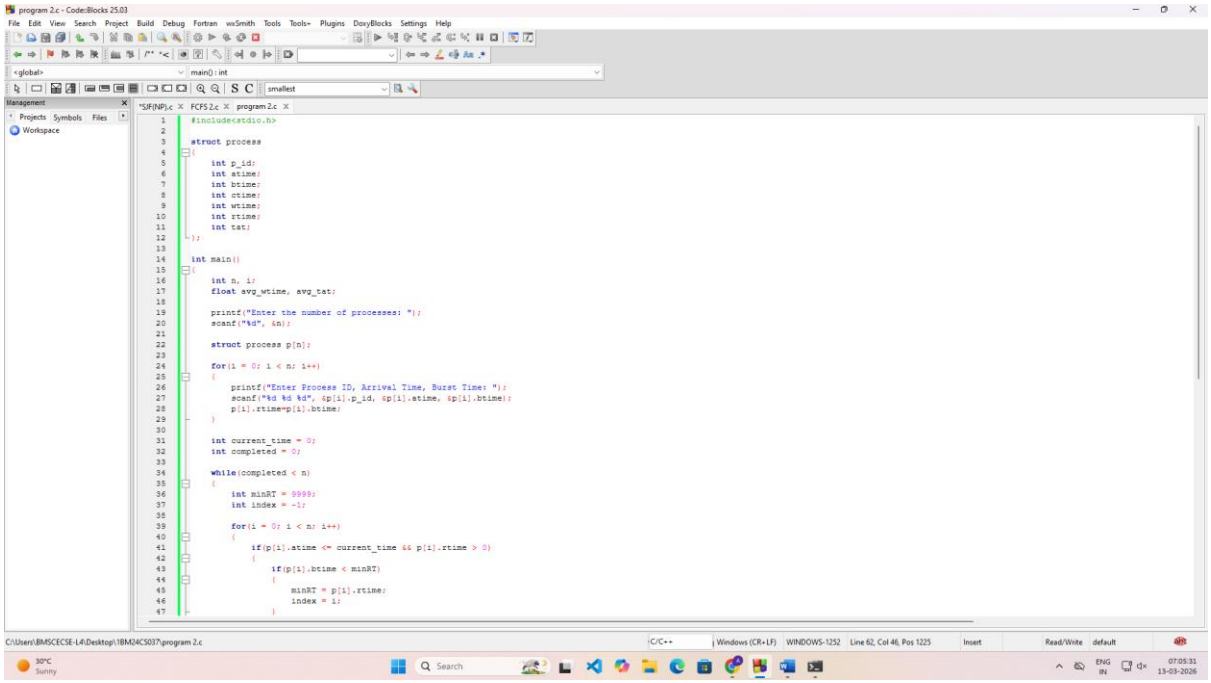
```c
#include<stdio.h>

struct process
{
    int atime;
    int btime;
    int p_id;
    int ctime;
    int wtime;
    int tat;
    int pc;
};

int main()
{
    int n, i, minBT, index;
    float avg_wtime, avg_tat;
    printf("Enter the number of processes:");
    scanf("%d", &n);
    struct process p[n];
    for(i=0; i<n; i++)
    {
        printf("Enter process id, arrival time and burst time:");
        scanf("%d  %d  %d", &p[i].p_id, &p[i].atime, &p[i].btime);
        p[i].pc=0;
    }

    int current_time=0;
    int completed=0;

    while(completed < n)
    {
        minBT = 9999;
        index = -1;
        for(i=0; i<n; i++)
        {
            if(p[i].atime<=current_time && p[i].pc==0)
            {
                if(p[i].btime<minBT)
                {
                    minBT=p[i].btime;
                    index=i;
                }
            }
        }
        if(index == -1)
        {
```



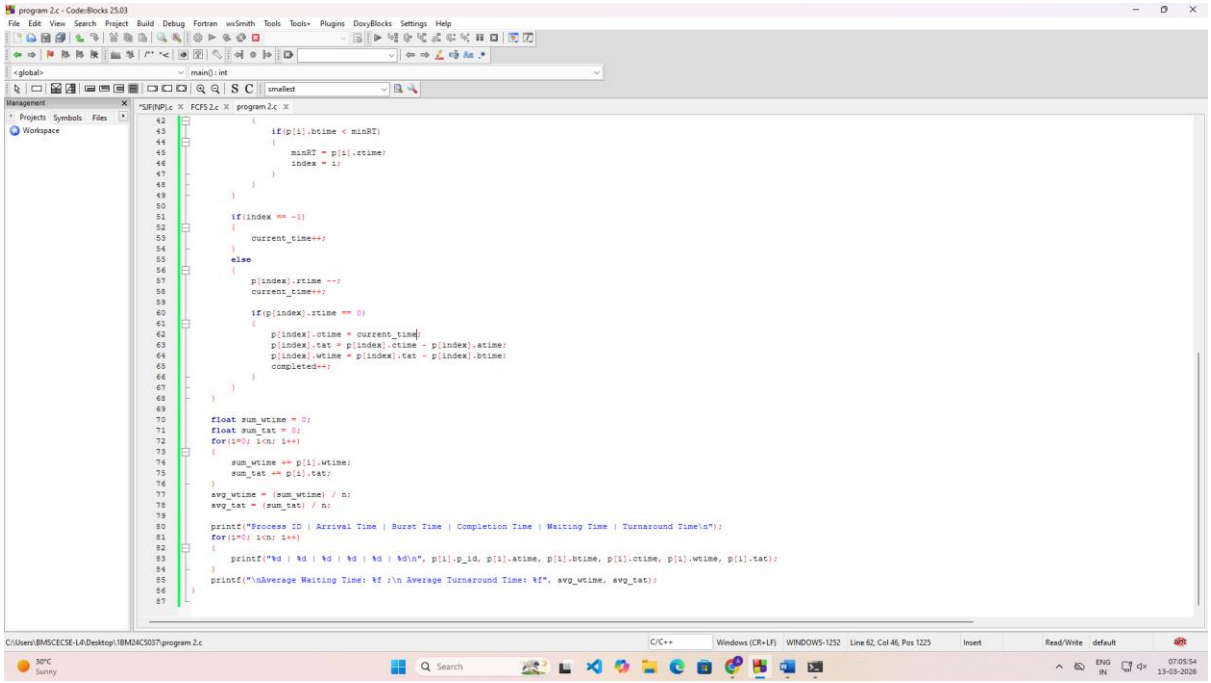
```c
    while(completed < n)
    {
        minBT = 9999;
        index = -1;
        for(i=0; i<n; i++)
        {
            if(p[i].atime<=current_time && p[i].pc==0)
            {
                if(p[i].btime<minBT)
                {
                    minBT=p[i].btime;
                    index=i;
                }
            }
        }
        if(index == -1)
        {
            current_time = current_time+1;
        }else{
            p[index].ctime = current_time + p[index].btime;
            p[index].tat = p[index].ctime - p[index].atime;
            p[index].wtime = p[index].tat - p[index].btime;
            current_time = p[index].ctime;
            p[index].pc=1;
            completed = completed+1;
        }
    }

    float sum_wtime = 0;
    float sum_tat = 0;
    for(i=0; i<n; i++)
    {
        sum_wtime += p[i].wtime;
        sum_tat += p[i].tat;
    }
    avg_wtime = (sum_wtime) / n;
    avg_tat = (sum_tat) / n;

    printf("Process ID | Arrival Time | Burst Time | Completion Time | Waiting Time | Turnaround Time\n");
    for(i=0; i<n; i++)
    {
        printf("%d | %d | %d | %d | %d | %d\n", p[i].p_id, p[i].atime, p[i].btime, p[i].ctime, p[i].wtime, p[i].tat);
    }
    printf("\nAverage Waiting Time: %f ;\n Average Turnaround Time: %f", avg_wtime, avg_tat);
}
```

Program 2: Shortest Job First (Non-Preemptive)

```
Enter the number of processes:4
Enter process id, arrival time and burst time:1 0 7
Enter process id, arrival time and burst time:2 2 4
Enter process id, arrival time and burst time:3 4 1
Enter process id, arrival time and burst time:4 5 4
Process ID | Arrival Time | Burst Time | Completion Time | Waiting Time | Turnaround Time
1 | 0 | 7 | 7 | 0 | 7
2 | 2 | 4 | 12 | 6 | 10
3 | 4 | 1 | 8 | 3 | 4
4 | 5 | 4 | 16 | 7 | 11

Average Waiting Time: 4.000000 ;
 Average Turnaround Time: 8.000000
Process returned 0 (0x0)   execution time : 39.280 s
Press any key to continue.
```



```c
#include<stdio.h>

struct process
{
    int p_id;
    int atime;
    int btime;
    int ctime;
    int wtime;
    int rtime;
    int tat;
};

int main()
{
    int n, i;
    float avg_wtime, avg_tat;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    struct process p[n];

    for(i = 0; i < n; i++)
    {
        printf("Enter Process ID, Arrival Time, Burst Time: ");
        scanf("%d %d %d", &p[i].p_id, &p[i].atime, &p[i].btime);
        p[i].rtime=p[i].btime;
    }

    int current_time = 0;
    int completed = 0;

    while(completed < n)
    {
        int minRT = 9999;
        int index = -1;

        for(i = 0; i < n; i++)
        {
            if(p[i].atime <= current_time && p[i].rtime > 0)
            {
                if(p[i].btime < minRT)
                {
                    minRT = p[i].rtime;
                    index = i;
                }
```



```c
            }
        }

        if(index == -1)
        {
            current_time++;
        }
        else
        {
            p[index].rtime --;
            current_time++;

            if(p[index].rtime == 0)
            {
                p[index].ctime = current_time;
                p[index].tat = p[index].ctime - p[index].atime;
                p[index].wtime = p[index].tat - p[index].btime;
                completed++;
            }
        }
    }

    float sum_wtime = 0;
    float sum_tat = 0;
    for(i=0; i<n; i++)
    {
        sum_wtime += p[i].wtime;
        sum_tat += p[i].tat;
    }
    avg_wtime = (sum_wtime) / n;
    avg_tat = (sum_tat) / n;

    printf("Process ID | Arrival Time | Burst Time | Completion Time | Waiting Time | Turnaround Time\n");
    for(i=0; i<n; i++)
    {
        printf("%d | %d | %d | %d | %d | %d\n", p[i].p_id, p[i].atime, p[i].btime, p[i].ctime, p[i].wtime, p[i].tat);
    }
    printf("\nAverage Waiting Time: %f ;\n Average Turnaround Time: %f", avg_wtime, avg_tat);
}
```

Extra: Shortest Job First (Preemptive)

```
Enter the number of processes: 3
Enter Process ID, Arrival Time, Burst Time: 1 0 6
Enter Process ID, Arrival Time, Burst Time: 2 1 3
Enter Process ID, Arrival Time, Burst Time: 3 2 7
Process ID | Arrival Time | Burst Time | Completion Time | Waiting Time | Turnaround Time
1 | 0 | 6 | 9 | 3 | 9
2 | 1 | 3 | 4 | 0 | 3
3 | 2 | 7 | 16 | 7 | 14

Average Waiting Time: 3.333333 ;
 Average Turnaround Time: 8.666667
Process returned 0 (0x0)   execution time : 16.742 s
Press any key to continue.
```